15,471,903 members Sign in

CODE PROJECT For those who code

articles Q&A forums stuff lounge ? Search for articles, questions,

EnumButtonGroup: An Alternative for Binding Enumerated Values to Buttons

Adrian Cole

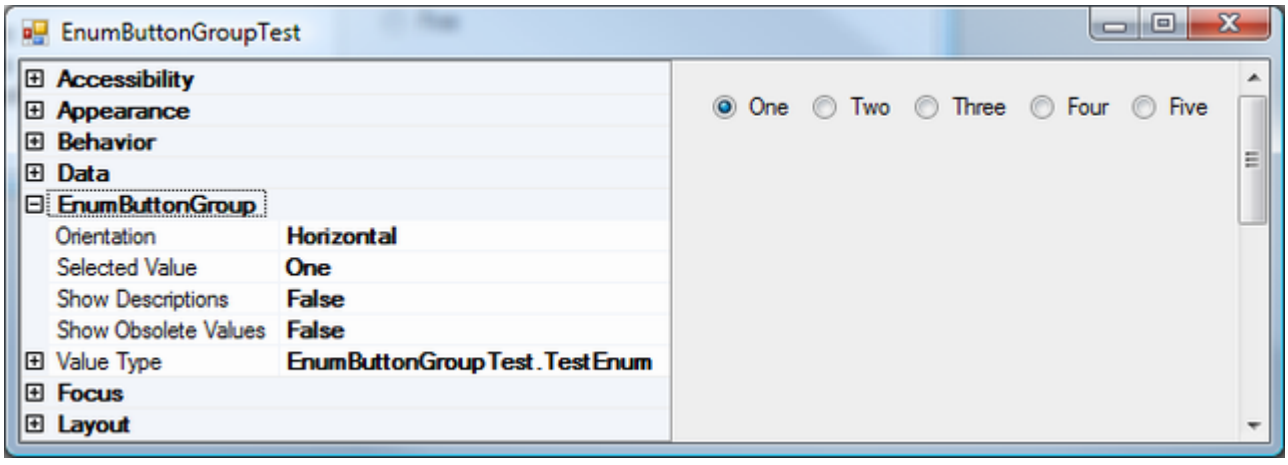
12 Dec 2008 CPOL

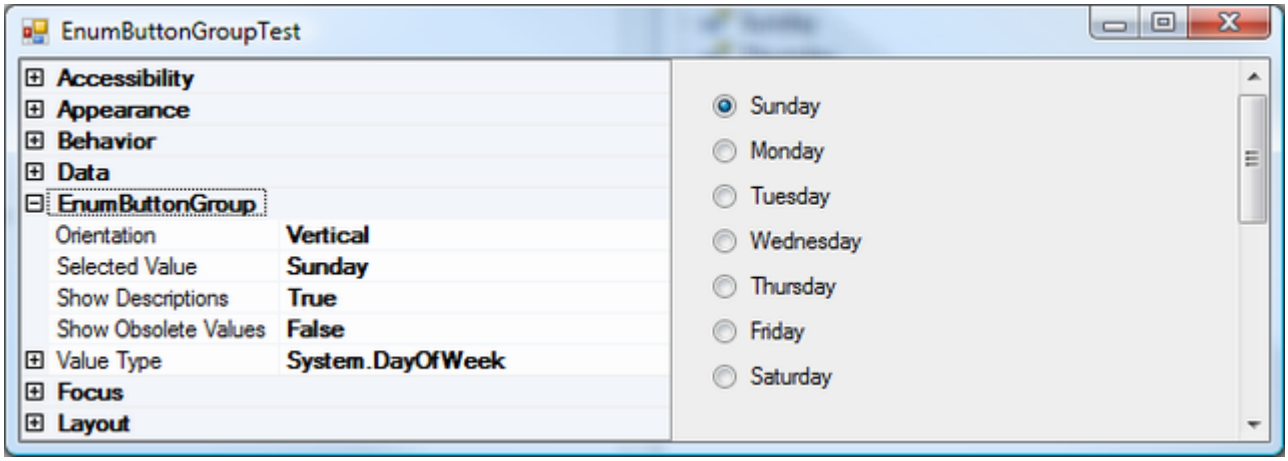
Rate me: 4.47/5 (7 votes)

A Windows Forms user-control that allows binding of enumerated values to a group of buttons, both at design-time and run-time.

Download source VS 2005 - 16.84 KB

Download source VS 2008 - 18.67 KB





Introduction

You can do an awful lot of cool stuff with the Windows Forms controls that come bundled with Visual Studio. Unfortunately, the binding of enumerated values to a group of buttons isn't one of them. This article introduces a control designed to do just that.

Background

I was slapping together a quick prototype at work the other day when I realized what I need was a control that I could use to display enumerated values as radio buttons. I didn't remember there being anything built into Visual Studio that would do the trick, so I did a quick online search. The closest thing I found to what I was looking for was this article by Jay Andrew Allen. However, his control didn't allow me to specify the enumerated type I wanted to display and have the control automatically create the buttons for me. A couple of hours later, I had created the control I wanted.

Using the Code

The `EnumButtonGroup` control is a Windows Forms `UserControl` that exposes the additional properties `ValueType` and `SelectedValue` and a `SelectedValueChanged` event.

The `ValueType` property is used to specify the .NET `Type` that represents the enumerated values that will be displayed in the control.

C# Shrink ▲

```
[TypeConverter(typeof(StringToTypeConverter))]
public Type ValueType
{
    get
    {
        return (this.valueType);
    }
```

```
    set
    {
        // Make sure value is an enumerated type
        if (value != null && !value.IsEnum)
        {
            throw new ArgumentException("Argument must be an enumerated type");
        }

        this.valueType = value;
        if (this.valueType == null)
        {
            // Reset
            this.Controls.Clear();
            this.Controls.Add(this.placeholderLabel);
            this.selectedValue = null;
            RaiseSelectedValueChangedEvent();
        }
        else
        {
            // Create button controls based on new valueType
            CreateButtons();
        }
    }
}
```

When this property is set, the control first checks to see if it represents an enumerated type. If so, it automatically creates the buttons that represent the values of that enumerated type. The `SelectedValue` property is then set to the first enumerated value and any subscribers are notified via the `SelectedValueChanged` event.

The `SelectedValue` property is used to specify which of the enumerated values displayed in the control will be selected (checked).

C# Shrink ▲

```
[Editor(typeof(EnumButtonGroupSelectedValueEditor), typeof(UITypeEditor))]
public object SelectedValue
{
    get
    {
        return (this.selectedValue);
    }
    set
    {
        // Don't bother setting if valueType is unknown
        if (this.valueType != null)
        {
            this.selectedValue = value;

            // If we are here as a result of a button CheckedChanged event
            // then there is no need to check the button
            if (!this.alreadyChecked)
            {
                CheckButton();
            }
```

```
            else
            {
                // Tell everyone the selected value has changed
                RaiseSelectedValueChangedEvent();
            }
        }
    }
}
```

When the `SelectedValue` property is set automatically in response to a `CheckChanged` event from one of the buttons, the control simply notifies any subscribers of the change via the `SelectedValueChanged` event. If the property is set directly then the appropriate button's `Checked` property is set to `true`, which in turn causes the `SelectedValueChanged` event to be raised.

Applications that want to be notified when the `SelectedValue` property changes should register to receive `SelectedValueChanged` events.

C#

```
public void MainForm_Load(object sender, EventArgs e)
{
    this.enumButtonGroup1.SelectedValueChanged +=
        new EventHandler(enumButtonGroup1_SelectedValueChanged);
}

...

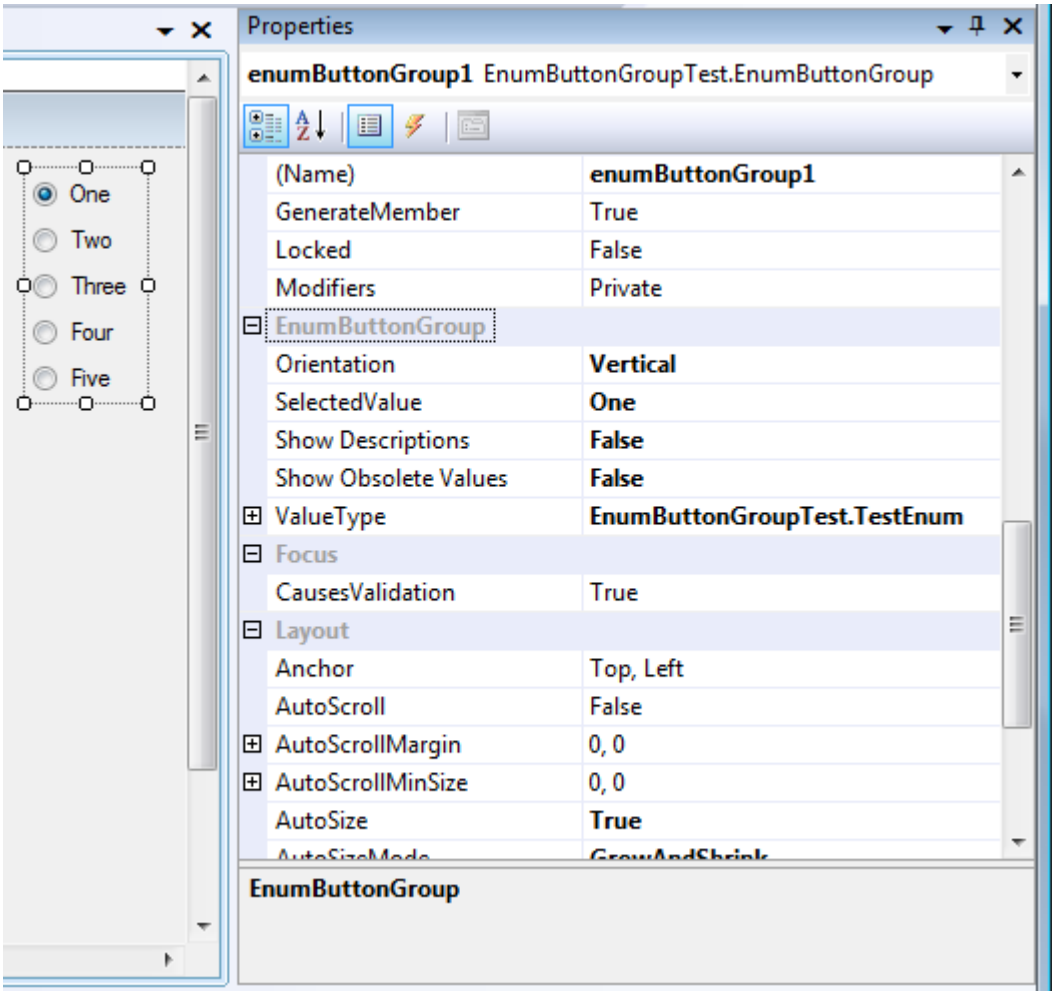
private void enumButtonGroup1_SelectedValueChanged(object sender, EventArgs e)
{
    this.propertyGrid1.SelectedObject = sender;
}
```

Design-Time Considerations

I wanted the `ValueType` property to be editable at design-time as well as run-time. To enable this, I needed to add a type converter (`StringToTypeConverter`) to convert a `string` to a `Type`, allowing `ValueType` to be specified in `string` format.

One point of special note: The `Type.GetType()` method will return `null` if the type isn't in an assembly loaded by the application or in the *mscorlib.dll* assembly.

I also wanted the `SelectedValue` property to be editable at design-time. This meant I needed to add a UI type editor (`EnumButtonGroupSelectedValueEditor`) to display drop-down list of possible values from which to select.



Points of Interest

You may have noticed that I also added some other properties:

- `Orientation` is an enumerated value that allows the buttons to be oriented either horizontally or vertically within the control
- `ShowDescriptions` is a boolean property that controls whether enumerated values marked with the `DescriptionAttribute` will have their description displayed
- `HideObsoleteValues` is a boolean property that controls whether enumerated values marked with the `ObsoleteAttribute` will be displayed

Some possible additions that would make this control more useful are:

- Specifying different layouts that would allow the buttons to be arranged in rows and columns
- Displaying enumerated values marked with the `FlagsAttribute` as check boxes

The attached project is a Visual Studio 2008 / .NET 3.5 project. If anyone would like a Visual Studio 2005 / .NET 2.0 version, please let me know and I'll upload one.

History

- 6th December, 2008: Initial submission
- 9th December, 2008: Added Visual Studio 2005 version. Renamed `ShowObsoleteValues` to `HideObsoleteValues` to correct inverse logic bug.

This is my first article submission. I will appreciate any feedback.

License

This article, along with any associated source code and files, is licensed under The Code Project Open License (CPOL)

Written By

Adrian Cole

Software Developer (Senior)

Canada

This member has not yet provided a Biography. Assume it's interesting and varied, and probably something to do with programming.

Comments and Discussions

You must Sign In to use this message board.

Search Comments

First Prev Next

Binding Enum values to buttons
Fregate 15-Dec-08 13:59

2005 Version Would be Nice
Preston McCormick 9-Dec-08 12:23

Re: 2005 Version Would be Nice

Adrian Cole 9-Dec-08 12:53

Ok, cool. But what about i18n

Robert Köpferl 8-Dec-08 11:36

From first impression: I like that, cool.
Maybe I did not quite understand. But seems like you display those strings you get from the enum{} type definition.
so what about localized apps. Is that possible?

Sign In · View Thread

Re: Ok, cool. But what about i18n
Adrian Cole 8-Dec-08 12:03

Re: Ok, cool. But what about i18n
Preston McCormick 10-Dec-08 4:47

Refresh 1

General News Suggestion Question Bug Answer Joke Praise Rant Admin

Use Ctrl+Left/Right to switch messages, Ctrl+Up/Down to switch threads, Ctrl+Shift+Left/Right to switch pages.

Permalink
Advertise
Privacy
Cookies
Terms of Use

Layout: fixed | fluid

Article Copyright 2008 by Adrian Cole
Everything else Copyright © CodeProject, 1999-2022

Web03 2.8:2022-10-01:1